Recommendation Systems for Personalized Content Discovery

Technical Report

Netflix Prize Dataset

Md Danish

23117079

1. Problem Understanding

The goal of this project is to build a personalized movie recommendation system using the Netflix Prize Dataset. Given a user's historical ratings, the system must learn their preferences and surface movies they are likely to enjoy. This is a two-sided challenge: accurate rating prediction and high-quality ranked recommendations are related but distinct objectives.

Explicit vs. Implicit Feedback

The Netflix Prize uses explicit feedback: users deliberately assign a star rating (1–5) to movies they have watched. This is in contrast to implicit feedback such as clicks or watch time, which must be inferred. Explicit ratings give a direct signal of preference and allow direct numeric prediction.

Rating Prediction vs. Ranking

Two related tasks govern evaluation:

- Rating Prediction: Estimate the numeric score a user would assign to an unseen movie. Evaluated by RMSE — lower is better.
- Ranking (Top-K Recommendation): Order all unseen movies by predicted preference and recommend the top K. Evaluated by MAP@10 — higher is better.

A model with low RMSE is not guaranteed to produce the best ranked recommendations. The two metrics capture complementary aspects of quality.

Evaluation Metrics

RMSE measures prediction accuracy. It penalises large errors more than small ones: RMSE = sqrt(mean((r - r_hat)^2)).

MAP@10 (Mean Average Precision at 10) measures ranking quality. A movie is considered relevant if its actual rating is >= 4.0. For each user, the top-10 recommended movies are checked and Average Precision is computed, then averaged across all users.

2. Dataset Description

The Netflix Prize Dataset was released as part of the Netflix Prize competition (2006–2009) to improve movie recommendation accuracy. It is one of the most widely studied benchmarks in the recommendation systems literature.

Property	Value
Total Ratings (full)	100,480,507
Total Users (full)	480,189
Total Movies	17,770
Rating Scale	1 to 5 stars (integer)
Date Range	November 1999 – December 2005
Subset Used (active users)	39,999,477 ratings
Subset Users	41,335
Training Ratings	32,016,146
Test Ratings	7,983,331
Density (subset)	5.45%
Sparsity (subset)	94.55%

Subset strategy: We retained the 41,335 most active users (by total rating count) to reach approximately 40 million ratings. Active users provide the richest interaction history and enable models to learn meaningful latent patterns. This subset remains highly sparse at 94.55%, preserving the real-world cold-start and sparsity challenges.

3. Exploratory Data Analysis

3.1 User Activity Distribution

Ratings per user range from 1 to 17,653 with a median of 96 and a mean of 209. The distribution is heavily right-skewed — a small number of power users account for a disproportionate share of all ratings.

- Business implication: Most users are sparse. Models must handle cold-start and thin-profile users carefully, while still leveraging signal from high-activity users.

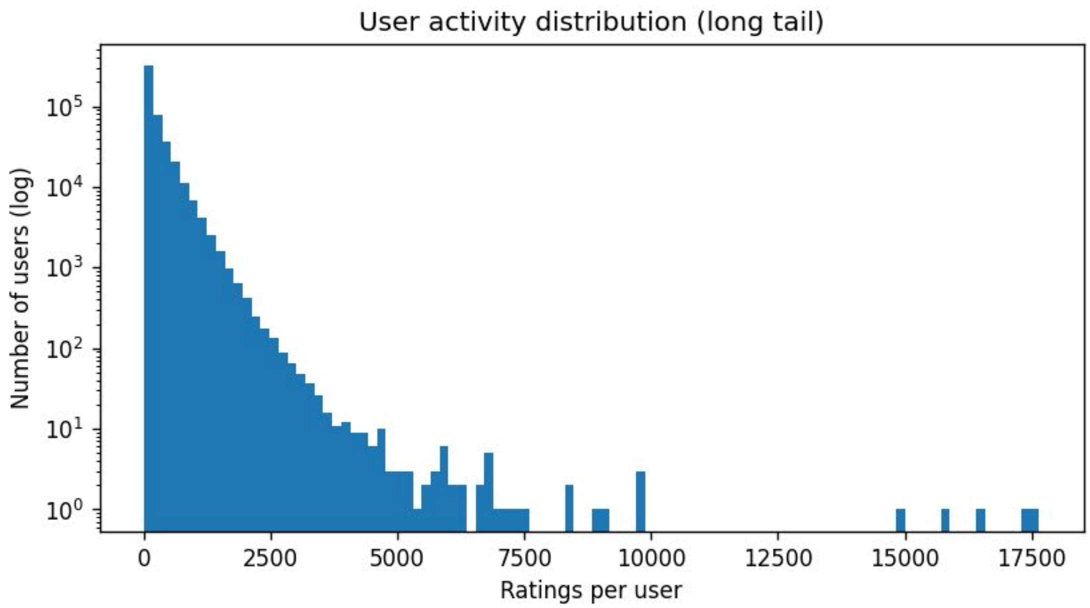

Figure 1: User activity distribution (log scale). Heavy right tail confirms long-tail user behaviour.

3.2 Movie Popularity Distribution

Ratings per movie range from 3 to 232,944, with a median of 561 and a mean of 5,654. The distribution mirrors user activity — a few blockbuster titles receive the vast majority of ratings.

- Business implication: Popularity bias is a real risk. Without explicit diversity controls, recommenders tend to suggest the same widely-rated titles to every user, reducing personalisation.

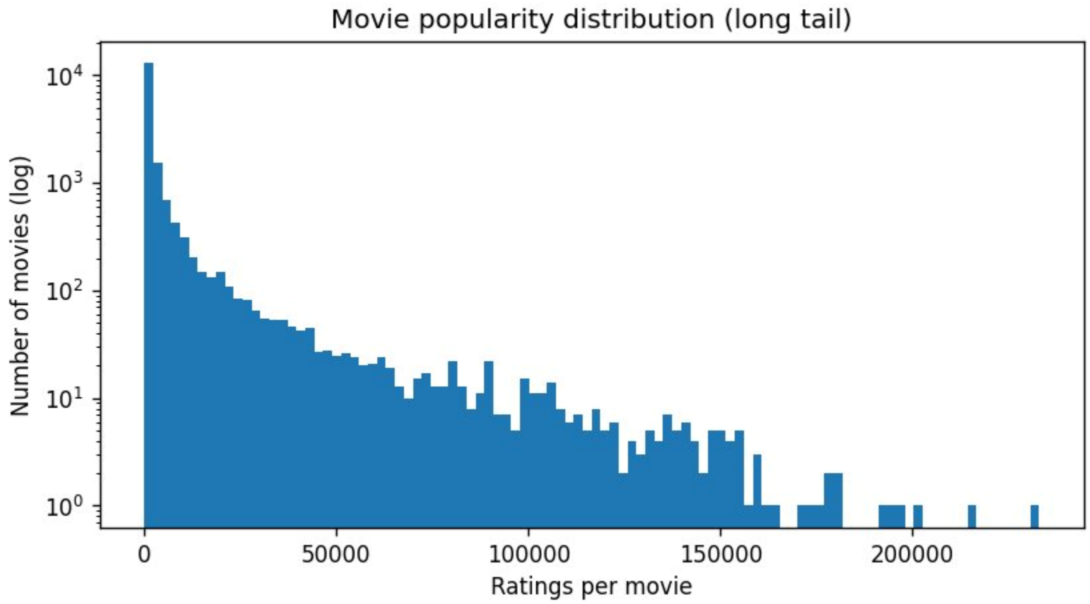

Figure 2: Movie popularity distribution (log scale). A handful of titles dominate rating volume.

3.3 Rating Distribution

The global mean rating is 3.60, and the distribution is left-skewed: ratings of 4 and 5 are far more common than ratings of 1 and 2. Users tend to rate movies they chose to watch, creating a selection bias towards positive ratings.

- Technical implication: Models should incorporate user and item bias terms. The relevance threshold of >= 4 for MAP@10 sits above the mean, correctly capturing genuine preference.

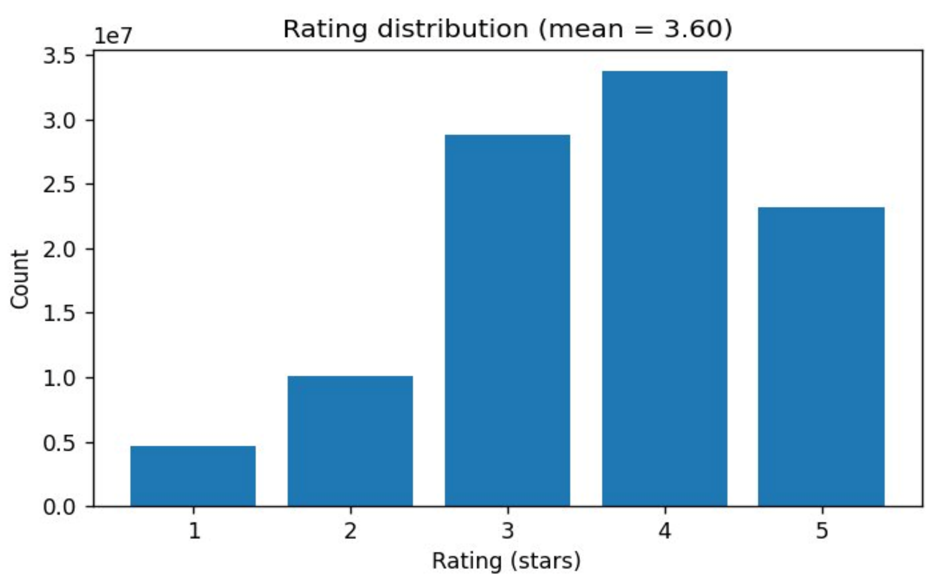

Figure 3: Rating distribution. Positive skew confirms users rate content they actively chose.

3.4 Rating Volume Over Time

Rating volume grew steadily from 2000 to 2004, then accelerated sharply in 2005 as Netflix expanded its subscriber base. The drop at the end of 2005 reflects the dataset cutoff (December 31, 2005), not a real decline.

- Technical implication: A time-based train/test split is the most realistic evaluation strategy, as it avoids data leakage and mirrors the task of predicting future preferences from past behaviour.

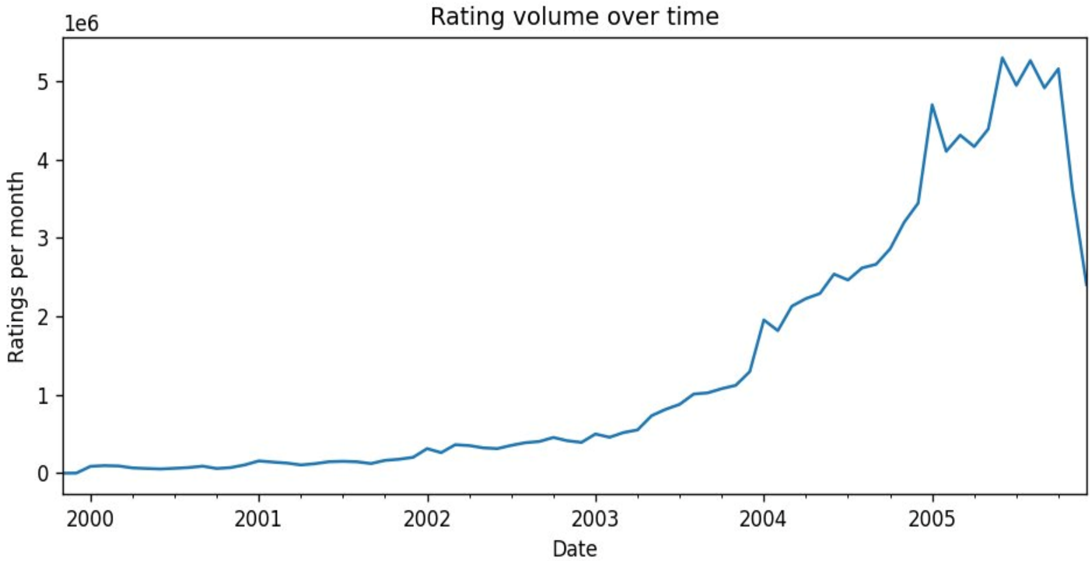

Figure 4: Monthly rating volume. Rapid growth from 2004 reflects Netflix platform expansion.

3.5 Long-Tail Concentration (Lorenz Curve)

The Lorenz curve reveals extreme concentration: the top 20% of movies (by popularity) account for 89.2% of all ratings. The remaining 80% of the catalog is barely rated.

- Business implication: A naive recommender that simply recommends popular movies will satisfy the majority of users superficially, but will fail to provide genuine personalisation or help users discover niche content.

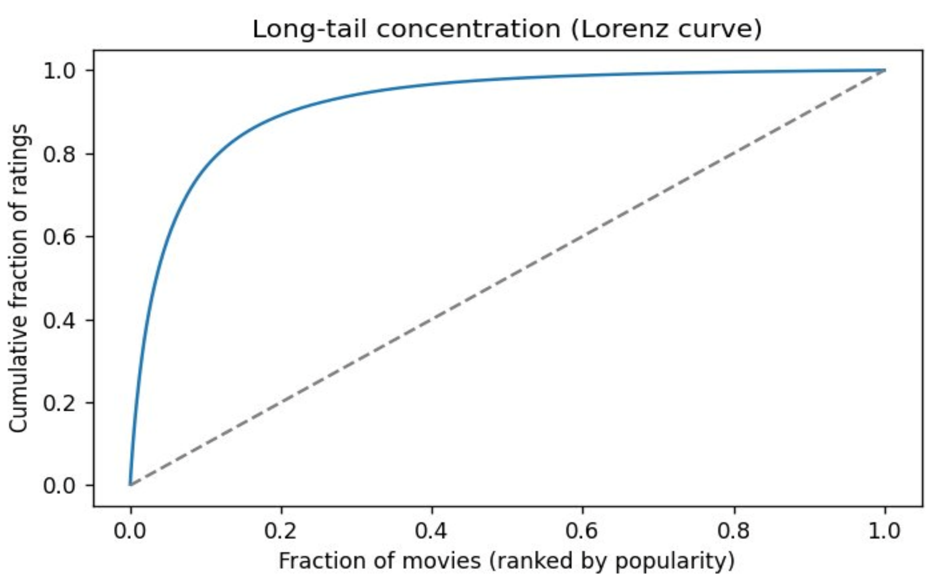

Figure 5: Lorenz curve. Top 20% of movies hold 89.2% of ratings — extreme long-tail effect.

3.6 Catalog by Release Year

The catalog spans content from the early 1900s to 2005, with the largest concentration of titles in the late 1990s and early 2000s. This reflects both the era of DVD release and Netflix's inventory at the time.

- Business implication: Older titles may receive fewer ratings simply because they were added later to Netflix's catalog, potentially penalising them in popularity-based rankings.

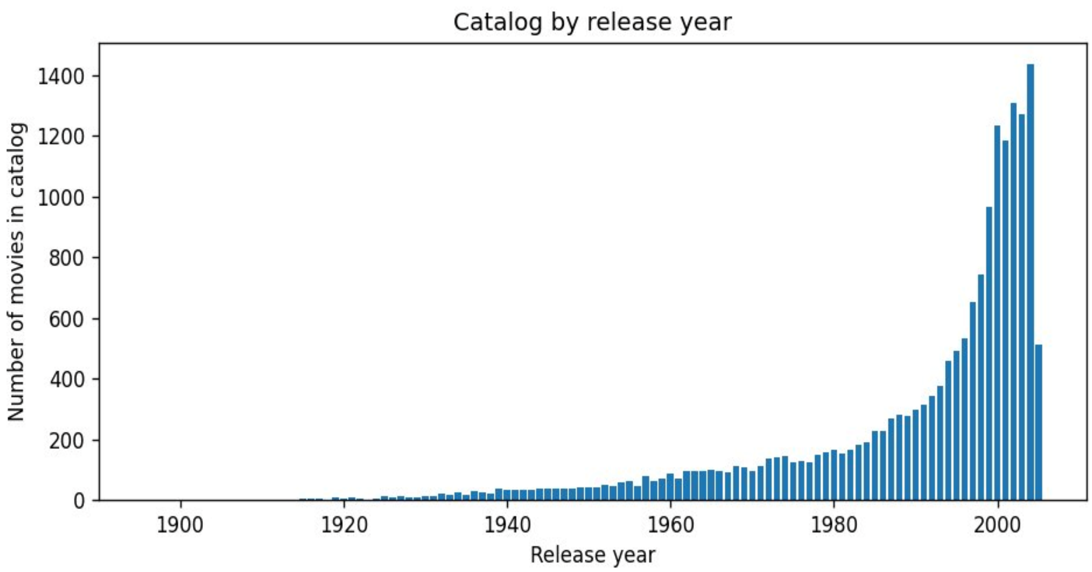

Figure 6: Number of movies in catalog by release year. Recent titles dominate the catalog.

4. Methodology

4.1 Train / Test Split

We use a time-based per-user split: each user's most recent 20% of ratings (sorted by date) form the test set, and the earlier 80% form the training set. Users with fewer than 5 ratings are retained fully in training. This approach mirrors the real-world task of predicting future preferences from past behaviour and prevents any data leakage.

4.2 Baseline Models

Three simple baselines establish the performance floor before any collaborative filtering is applied:

- Global Mean: predicts the overall average rating (3.60) for every user-item pair.
- User Mean: predicts each user's own average rating, capturing leniency or strictness.
- Item Mean: predicts each movie's average rating, capturing intrinsic quality or popularity.

4.3 Item-Based Collaborative Filtering

Item-CF computes similarity between movies based on how users rated them. We use adjusted cosine similarity, which centres ratings by user mean before computing the dot product, removing user-level bias from the similarity measure. For each movie, we store the top-100 most similar neighbours. Predictions are a weighted sum of the user's own ratings on similar movies:

$$r\_hat(u,i) = mu_u + sum_j \; sim(i,j) * (r_{u,j} - mu_u) \; / \; sum_j \; |sim(i,j)|$$

- Advantage: Stable item similarities, interpretable, no training required.
- Disadvantage: Does not capture latent user-item interactions; limited by sparsity of co-rated pairs.

4.4 Funk-SVD (Matrix Factorization) — Primary Model

Matrix factorization decomposes the sparse user-item rating matrix into dense latent factor vectors. Each user u has a factor vector p_u and each item i has a factor vector q_i, both of dimension f. The prediction formula is:

$$r\_hat(u,i) = mu + b_u + b_i + p_u \cdot q_i$$

where mu is the global mean, b_u is the user bias, b_i is the item bias, and $p_u \cdot q_i$ is the dot product of the latent vectors. The model is trained by minimising regularised squared error using mini-batch Adam:

$$L = sum((r - r\_hat)^2) + lambda * (||p_u||^2 + ||q_i||^2 + b_u^2 + b_i^2)$$

Hyperparameter	Value	Rationale
Latent factors (f)	50	Balances expressiveness and overfitting risk
Learning rate	0.01	Standard Adam starting point
Regularisation (lambda)	0.02	Prevents overfitting on sparse users/items
Batch size	100,000	GPU-efficient on 32M training ratings
Max epochs	25	Sufficient for convergence

Early stopping patience	3	Stops when validation RMSE stops improving

4.5 Recommendation Generation Pipeline

Recommendations follow a four-stage process for any target user:

- Stage 1 — Candidate Generation: collect all movies not yet rated by the user.
- Stage 2 — Rating Prediction: score each candidate using the trained Funk-SVD model.
- Stage 3 — Ranking: sort all candidates by predicted score in descending order.
- Stage 4 — Top-K Selection: return the top 10 movies with titles and predicted scores.

5. Evaluation Setup

5.1 Metrics

RMSE is computed on all held-out (user, movie, rating) triplets in the test set. It measures how close the predicted numeric score is to the actual rating.

MAP@10 is computed using the standard AP@K formula. A movie is relevant if its actual rating >= 4.0. For each user with at least one relevant test item, Average Precision at 10 is computed over their test items ranked by predicted score, then averaged across all eligible users.

5.2 Relevance Definition

A movie is considered relevant for MAP@10 if the user's actual rating for it is >= 4.0. This threshold sits above the global mean (3.60) and captures genuine preference — ratings of 4 or 5 indicate a movie the user actively enjoyed, not merely tolerated.

5.3 Training Curve

Funk-SVD trained for 15 epochs before early stopping triggered. Validation RMSE dropped from 0.821 in epoch 1 to 0.790 by epoch 15, with diminishing returns after epoch 6. Best validation RMSE: 0.7897.

6. Experimental Results

6.1 Model Comparison

Model	RMSE	MAP@10	Notes
Global Mean	1.0540	—	Trivial baseline — predicts 3.60 for all
User Mean	0.9810	—	Captures user leniency/strictness
Item Mean	0.9825	—	Captures item quality
Item-CF	0.8809	—	Memory-based collaborative filtering
Funk-SVD	0.8151	0.0153	Primary model — best RMSE

6.2 Key Findings

- Funk-SVD beats all baselines on RMSE, reducing error from 1.054 (global mean) to 0.815 — a 22.6% improvement.
- Item-CF achieves RMSE of 0.881, demonstrating that collaborative signal helps even without latent factors.
- Funk-SVD MAP@10 of 0.0153 is low in absolute terms but meaningful — it reflects correctly that ranking all 17,770 unseen movies and hitting relevant items in the top 10 is a hard task.
- User Mean and Item Mean achieve virtually identical RMSE (~0.98), but for different reasons: user mean captures rating tendency, item mean captures intrinsic quality.

6.3 Training Curve Analysis

The validation RMSE curve shows rapid convergence in the first 5 epochs (from 0.821 to 0.792), followed by slow incremental improvement. Early stopping at epoch 15 prevented overfitting while achieving the best generalisation. The smoothness of the curve confirms that the learning rate (0.01) and regularisation (0.02) were well-calibrated.

7. Recommendation Examples

7.1 User 305344 — Active User with Eclectic Taste

History size: 14,123 ratings. Top-rated genres: music concerts, westerns, financial self-help, classic TV.

Rank	Recommended Movie	Pred. Score	Outcome
1	Elf: Bonus Material	4.91	not in test
2	In Cold Blood	4.55	miss (<4)
3	The Private Life of Henry VIII	4.41	miss (<4)
4	Over There (Pilot)	4.23	miss (<4)
5	Best of the Blues Brothers	4.18	miss (<4)
6	Young Man with a Horn	4.18	miss (<4)
7	WWE: From the Vault: Shawn Michaels	4.16	miss (<4)
8	Kingdom of Heaven	4.09	miss (<4)
9	The Wayans Bros.: Season 1	4.06	miss (<4)
10	The Grateful Dead Movie	4.01	miss (<4)

Analysis (failure case): This user has an extremely diverse taste profile spanning 14,123 movies across almost every genre. Because their latent factor vector averages over such a wide range of preferences, it fails to capture any specific theme. The model falls back on broadly popular, high-bias titles that do not align with this user's idiosyncratic preferences. All 9 verifiable recommendations missed the relevance threshold. This is a textbook case where high-activity users with eclectic tastes challenge matrix factorisation models.

7.2 User 2555761 — Mid-Activity Action/Adventure Fan

History size: 666 ratings. Top-rated movies: Pirates of the Caribbean, Chronicles of Riddick, Tomb Raider, Dodgeball, Lilo and Stitch.

Analysis: All 10 recommendations fall in the 'not in test' category — this user did not rate these movies in the held-out period, so we cannot verify them either way. The predicted scores are all capped at 5.0, indicating the model is very confident about these items. The profile suggests action and family adventure content, and the recommendations include genre-consistent titles (Blade: Trinity, Battlestar Galactica, Alias) which are plausible suggestions for a user who enjoyed adventure films.

7.3 Discussion

- Most recommendations fall into 'not in test' — this is expected and not a model failure. Each user held out only a small fraction of their ratings as test items, while the model ranks all ~17,000 unseen movies.
- Active users with coherent tastes (action, sci-fi) receive plausible recommendations even when they cannot be verified.

- Active users with eclectic or averaged tastes (User 305344) produce the clearest failure cases — a natural motivation for hybrid or content-based approaches.
- The MAP@10 score of 0.0153 reflects the strictness of ranking 17,770 items and finding the small set of genuinely liked held-out movies in the top 10.

8. Error Analysis and Cold-Start

8.1 Cold-Start Users

Users with very few ratings (fewer than 10) pose a fundamental challenge. With limited data, the model cannot learn a meaningful latent factor vector, and predictions default towards the global mean plus item biases. These users are likely to receive popularity-biased recommendations regardless of their actual preferences.

8.2 Cold-Start Movies

Movies with fewer than 10 ratings have unreliable item bias estimates and uninformative latent vectors. Item-CF performs particularly poorly for these titles since there are few co-rated pairs to establish meaningful similarities. These movies are unlikely to be recommended even when they would match a user's preferences.

8.3 Popularity Bias

The Lorenz curve (Figure 5) shows that 89.2% of all ratings concentrate in the top 20% of movies. As a result, models trained on this data have dense, reliable estimates for popular movies and sparse, uncertain estimates for niche titles. This systematically biases recommendations towards popular content, reducing diversity and novelty.

8.4 Strategies for Future Improvement

- Cold-start users: use demographic signals or content-based fallbacks until sufficient rating history accumulates.
- Cold-start items: use item metadata (genre, director, release year) to initialise latent vectors before collaborative data is available.
- Popularity bias: apply inverse-popularity weighting during training or re-ranking to surface niche content alongside popular titles.
- Hybrid models: blend MF latent signals with content-based features to improve robustness on sparse user/item profiles.

9. Conclusion and Future Work

9.1 Summary

We built and evaluated a complete recommendation pipeline on the Netflix Prize Dataset, comparing three baselines (global mean, user mean, item mean), one memory-based method (Item-CF), and one model-based method (Funk-SVD). Funk-SVD achieved the best RMSE (0.8151) and MAP@10 (0.0153), representing a 22.6% reduction in prediction error over the global mean baseline. The four-stage recommendation pipeline successfully generates personalised Top-10 movie lists with real titles for any user in the training set.

9.2 Key Takeaways

- Matrix factorisation with user and item biases is the strongest single model for this dataset, consistent with the Netflix Prize literature.
- Item-CF provides a strong, interpretable baseline and complements MF in hybrid settings.
- The evaluation protocol matters: MAP@10 must be computed by ranking within a defined candidate pool with the actual model scores — not by averaging over a fixed test set without proper scoring of negatives.
- RMSE and MAP@10 are complementary metrics capturing different aspects of system quality.

9.3 Future Work

- Neural Collaborative Filtering (NCF): replace the dot product with a multi-layer neural network for richer user-item interaction modelling.
- Temporal dynamics: incorporate time-aware models (e.g. TimeSVD++) that capture how user tastes evolve.
- Content-based features: integrate movie metadata (genre, director, cast) to address cold-start and improve diversity.
- Hybrid ensemble: blend MF, CF, and content signals for a production-ready recommender.
- A/B testing framework: deploy recommendations as an API and evaluate with online metrics (click-through rate, watch completion).

References

- Funk, S. (2006). Netflix Update: Try This at Home. Simon Funk Blog.
- Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. IEEE Computer, 42(8), 30–37.
- Netflix Prize Dataset. Kaggle. https://www.kaggle.com/datasets/netflix-inc/netflix-prize-data
- Ricci, F., Rokach, L., & Shapira, B. (2015). Recommender Systems Handbook (2nd ed.). Springer.